Experiment No. 7:
Arrays in C – Part I

Objectives

The objectives of this lab are:

1. To learn the declaration, initialization and manipulation of arrays in C.
2. To implement 1-D arrays in solving computational problems.
3. To understand sorting and searching in arrays.
4. To learn about static local arrays and automatic local arrays.

Prerequisites

Following are the prerequisites of this lab are:

1. Understanding of data types in C programming language.
2. Basic knowledge of variables, loops, control statements and functions in C.
3. Read chapter 6 up to section 6.8 from textbook (C-How to Program, 9 Ed.)

Introduction

Array
An array stores multiple values of same data type under a single named variable. All elements of an array are stored contiguously in the memory.

Defining and initialization of an Array
An array is defined by declaring its data type, name, and size (number of elements). Initialization of an array means assigning value to its element. An array can be initialized at the time of declaration or later by assigning values to individual elements using their index positions.
Such as;

DataType arrayName[arraySize] ➡ int arr[5];

Syntax for initialization of an Array

```
//Deceleration of an Array
DataType arrayName[arraySize];

//Deceleration &Initialization of an Array at Same Time
DataType arrayName[arraySize]= {value1, value2, value3, ..., valueM};
```

Initialization of an Array Using Index Position

```
DataType arrayName[arraySize];

arrayName[0]= value1;
arrayName[1]= value2;
arrayName[2]= value3;
...
arrayName[N]= valueM;
```

Example:

```c
#include <stdio.h>
// Defining an Array to Display Elements
int main() {
    int numbers[5] = {10, 20, 30, 40, 50};
    printf("Array elements are:\n");
    printf("numbers[0] = %d\n", numbers[0]);
    printf("numbers[1] = %d\n", numbers[1]);
    printf("numbers[2] = %d\n", numbers[2]);
    printf("numbers[3] = %d\n", numbers[3]);
    printf("numbers[4] = %d\n", numbers[4]);
    return 0;
}
```

Output:

```
muhammad-umar@umarrsthinkpad:~/Desktop/PF_Labs/Lab07_2025_EE_016$ ./output1
Array elements are:
numbers[0] = 10
numbers[1] = 20
numbers[2] = 30
numbers[3] = 40
numbers[4] = 50
muhammad-umar@umarrsthinkpad:~/Desktop/PF_Labs/Lab07_2025_EE_016$
```

Note: In C, the brackets [ ] used to access array elements have the highest precedence among operators. Array subscripts are processed before almost all other operations. As mentioned in a table of section 6.2 of your textbook.

Concept of 1-D Array

A one-dimensional (1-D) array is a linear list of elements of the same data type. Each element is accessed using its index, where the first element is at index 0 and the last is at size – 1. A 1-D array is ideal for storing sequential data such as marks, temperatures, readings, or counters.
Because the array elements are placed in continuous memory, the CPU can calculate the address of any element efficiently using:

Address = Base Address + (Index × Size of Data Type)

This is illustrated in the following C language code;

C Code

```
#include <stdio.h>

int main() {
    int arr[5] = {10, 20, 30, 40, 50};

    printf("Base address (arr) = %p\n", arr);
    printf("Address of arr[1] = %p\n", &arr[1]);
    printf("Address of arr[2] = %p\n", &arr[2]);
    return 0;
}
```

Output

```
Base address (arr) = 000000792F9FF920
Address of arr[1] = 000000792F9FF924   {base address + 1 x 4}
Address of arr[2] = 000000792F9FF928
```

Note: "%p" is a format specifier in C used inside printf( ) to print a memory address.

Using a Loop to Set the Array's Element

Since an array contains multiple elements and each is accessed using an index, we use a loop (usually a for loop) to access them in sequence. A loop runs from index 0 to size – 1 to traverse the value of array at each index.
C Code

```
#include <stdio.h>

int main() {
    int arr[5] = {10, 20, 30, 40, 50};   // declaring and initializing an array
```

```
int i;

    printf("Traversing Array Elements:\n");

    // Traversal using a for loop
    for (i = 0; i < 5; i++) {
        printf("Element at index %d = %d\n", i, arr[i]);
    }

    return 0;
}
```

Output

```
Traversing Array Elements:
Element at index 0 = 10
Element at index 1 = 20
Element at index 2 = 30
Element at index 3 = 40
Element at index 4 = 50
```

Insertion of an Element in an Array

Insertion involves adding a new element at a specific position in the array by shifting elements to create space. Use a loop to start from the last element (index n-1) and go backwards to the position to create space for a new insertion.

C Code

```
#include <stdio.h>
int main() {
    // Initialize an array with 5 elements
    int arr[10] = {1, 2, 3, 4, 5};

    int n = 5;      // Current number of elements in the array
    int pos = 3;    // Position where we want to insert a new element (1-based index
    int value = 99; // Value to be inserted
    printf("Array before insertion: "); // Print the array before insertion
    for(int i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
```

```
}
    printf("\n");

    // Shift elements to the right to create space for the new element
    for(int i = n; i >= pos; i--)
    {
        arr[i] = arr[i-1]; // Move each element one position to the right
    }
    arr[pos-1] = value; // Insert the new value at the specified position
    n++; // Increase the total number of elements in the array

    printf("Array after insertion: "); // Print the array after insertion
    for(int i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }
    return 0;
}
```

Output

```
Array before insertion: 1 2 3 4 5
Array after insertion: 1 2 99 3 4 5
```

Deletion of an Element from an Array

To delete an element, shift the remaining elements left to fill the position. This effectively overwrites the element that would be deleted. It does not reduce the size of an array, it just shows that the array has now one less element.

C Code

```
#include <stdio.h>
int main() {
    int arr[10] = {1, 2, 3, 4, 5};  // Initial array
    int n = 5;                       // Number of elements in the array
    int pos, i;

    printf("Array before deletion: ");
    for(i = 0; i < n; i++)
```

```
{
        printf("%d ", arr[i]);
    }
    printf("\n");

    printf("Enter the position of the element to delete (0 to %d): ", n-1);
    scanf("%d", &pos);

    if(pos < 0 || pos >= n) {
        printf("Invalid position!\n");
    } else {
        // Shift elements to the left
        for(i = pos; i < n-1; i++) {
            arr[i] = arr[i+1];
        }
        n--; // Reduce array size

        printf("Array after deletion: ");
        for(i = 0; i < n; i++)
        {
            printf("%d ", arr[i]);
        }
        printf("\n");
    }
    return 0;
}
```

Output

```
Array before deletion: 1 2 3 4 5
Enter the position of the element to delete (0 to 4): 3
Array after deletion: 1 2 3 5
```

Length of an Array in C

The Length of an array in C refers to the maximum number of elements that an array can hold. It is also known as the size of an array that is used to determine the memory required to store all of its elements. The simplest method to find the length of an array is by using sizeof( ) operator to get the total size of the array and then divide it by the size of one element.

C Code

```
#include <stdio.h>
int main() {
    int arr[5] = { 1, 2, 3, 4, 5 };

    // Find the size of the array arr
    int u = sizeof(arr); // Gives the size of tota memory for array in bytes.
    int n = sizeof(arr) / sizeof(arr[0]);// Gives total number of elements in array

    printf("Memory size occupied by the array is %d bytes.\n", u);
    printf("The size of the array is %d.", n);
    return 0;
}
```

Output:

```
Memory size occupied by the array is 20 bytes.
The size of the array is 5.
```

Note: **"sizeof(arr)"** gives the total memory occupied by the entire array, not the number of elements. If you only used **"sizeof(arr)"**, you would get the total size in bytes, not how many elements there are.

Avoid Mixing Data Types in Arrays

It is important to note that all elements in an array must be of the same data type. This means you cannot mix different types of values, like integers and floating-point numbers, in the same array. For example;

```
int numbers[3] = {10, 20, 30};        // Valid Deceleration
float scores[3] = {10.5, 20.0, 30.2}; // Valid Deceleration
char letters[3] = {'A', 'B', 'C'};    // Valid Deceleration
int Array[3] = {10, 20.5, 'A'};       // Invalid Deceleration
```

Passing Array to a Function

Passing an array to a function allows the function to directly access and modify the original array. Passing array elements to a function is similar to passing variables to a function. Passing an array to a function means to send its location in memory rather than sending all its data. The function goes to that location and works on the real array. That's why changes inside the function affect the original array. There are two ways to pass an array to a function, one is a direct method while in other method pointers are used. The topic of pointers will be discussed in detail in upcoming manuals, but here a small description is given to differentiate arrays from pointer.

Note: Array vs Pointer

The array name represents the starting address of the first element, but it is not a variable and cannot be changed. A pointer, on the other hand, is a special variable that stores the memory address of another variable or the first element of an array. Unlike an array name, a pointer can change to point to different memory locations. In simple terms, an array stores many values together, while a pointer only stores the address of a value.

Syntax for Passing Array to a Function

```
//Using array notation
returnType functionName(Datatype arrayName[], int size);

//Using pointer notation
returnType functionName(Datatype *arrayName, int size);
```

Note: C does not keep track of how many elements are inside the array. Therefore, the size must be passed as another argument.

C Code

```
// Program to calculate the sum of array elements by passing to a function
#include <stdio.h>
float calculateSum(float num[]);

int main() {
  float result, num[] = {23.4, 55, 22.6, 3, 40.5, 18};

  // num array is passed to calculateSum()
  result = calculateSum(num);
  printf("Total sum is = %.2f", result);
  return 0;
}

float calculateSum(float num[]) {
  float sum = 0.0;

  for (int i = 0; i < 6; ++i) {
    sum += num[i];
  }

  return sum;
}
```

Output

```
Total sum is = 162.50
```

Static Local Arrays and Automatic Local Arrays

Array initialization refers to assigning initial values to the elements of an array at the time of its declaration. This process can be done in two primary ways:

- **Static Initialization:** The array is initialized at compile-time.
- **Automatic Initialization:** The array is initialized at runtime, typically using loops.

In C programming, local arrays inside a function can be classified as static or automatic. Static local arrays are declared with the static keyword and retain their values between function calls because their memory is allocated in the data segment and exists for the lifetime of the program. In contrast, automatic local arrays are the standard arrays declared without static; they are created on the stack when the function is called and destroyed when the function exits, so they do not preserve values across calls.

C Code

```c
#include <stdio.h>

int staticArrayDemo() {
    static int arr[1] = {0};
    arr[0] += 1; // increment first element
    return arr[0];
}

int automaticArrayDemo() {
    int arr[1] = {0};
    arr[0] += 1; // increment first element
    return arr[0];
}

int main() {
    printf("Static: %d, Automatic: %d\n", staticArrayDemo(),
automaticArrayDemo());
    printf("Static: %d, Automatic: %d\n", staticArrayDemo(),
automaticArrayDemo());
    printf("Static: %d, Automatic: %d\n", staticArrayDemo(),
automaticArrayDemo());
    return 0;
}
```

Output

```
Static: 1, Automatic: 1
Static: 2, Automatic: 1
Static: 3, Automatic: 1
```

Note: Choosing between static and automatic arrays depends on whether the data needs to be remembered between multiple successive calls of the function.

Using Character Arrays to Store and Manipulate Strings

In C, strings are represented as arrays of characters ending with a special null character "\0" that marks the end of the string. Character arrays allow programmers to store, read, modify, and display textual data. A value can be directly assigned when declaring the array; moreover, individual characters can be changed, or can take input from the user. Proper handling of the null character is important to ensure that functions like printf() and scanf() work correctly.
C Code (Storing and Printing a String):

```c
#include <stdio.h>

int main() {
    char String[] = "Storing and printing a string";  // Character array
initialized
    printf("This code is about: %s\n", String);
    return 0;
}
```

Output

```
This code is about: Storing and printing a string
```

C Code (Modification in a String):

```c
#include <stdio.h>
//This code shows how to modify a string
int main() {
    char str[6] = "Hello";
    str[0] = 'J';  // Change first character
    printf("Modified string: %s\n", str); // Output: Jello
    return 0;
}
```

Output

```
Modified string: Jello
```

Character Array (String) Operations in C

- Finding Length (**strlen**): Returns the number of characters in a string (excluding \0).
- Concatenation (**strcat**): Appends one string to the end of another.
- Copying (**strcpy**): Copies the content of one string into another.
- Comparing (**strcmp**): To Compares two strings.
- Finding a Character (**strchr**): To find the first occurrence of a character in a string.
- Finding a Substring (**strstr**): To find the first occurrence of a substring in a string.

C Code

```c
#include <stdio.h>
#include <string.h>
int main() {
    char str1[50] = "Hello";
    char str2[] = " World";
    char str3[50];

    // Finding Length
    printf("Length of str1: %lu\n", strlen(str1));
    // Concatenation
    strcat(str1, str2);
    printf("Concatenated string: %s\n", str1);
    // Copying
    strcpy(str3, str1);
    printf("Copied string into str3: %s\n", str3);
    // Comparing
    int cmp = strcmp(str1, str3);
    printf("Comparison of str1 and str3: %d\n", cmp);
    // Finding a Character
    char *ch = strchr(str1, 'W');
    if(ch != NULL)
        printf("Character 'W' found at position: %ld\n", ch - str1 + 1);
    else
        printf("Character 'W' not found\n");
    // Finding a Substring
    char *sub = strstr(str1, "World");
    if(sub != NULL)
        printf("Substring 'World' found at position: %ld\n", sub - str1 + 1);
    else
        printf("Substring 'World' not found\n");
    return 0;
}
```

Note: In C, strings are **arrays of characters** ending with a null character \0. To perform operations like finding the length, concatenating, copying, or comparing strings, instead of writing manual code, it is convenient to use to use string.h header file, having predefined functions for all these operations.

Output

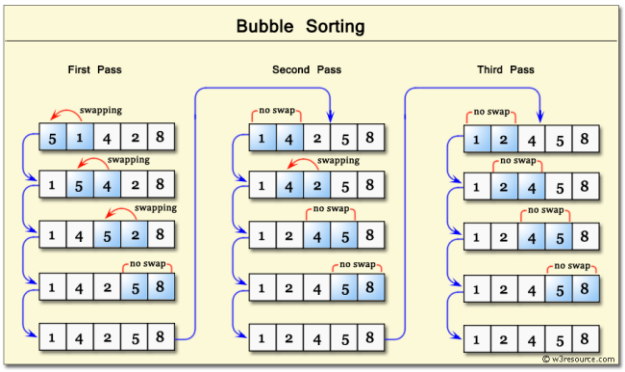
```
Length of str1: 5
Concatenated string: Hello World
Copied string into str3: Hello World
Comparison of str1 and str3: 0
Character 'W' found at position: 7
Substring 'World' found at position: 7
```

Sorting Array

Sorting is the process of arranging the elements of an array in a particular order, such as ascending or descending. Sorting is an essential operation in programming because it helps organize data, makes searching more efficient, and simplifies analysis and reporting. In C, arrays are often sorted using different algorithms depending on the size of the array and the desired efficiency.

Bubble Sort

Bubble Sort is a simple comparison-based sorting algorithm. In Bubble Sort, adjacent elements are compared, and they are swapped if they are in the wrong order. This process is repeated for all elements of the array until the array becomes fully sorted. This process of bubble sorting is illustrated below;



C Code

```
#include <stdio.h>
int main() {
    int arr[] = {5, 2, 9, 1, 5};
    int n = sizeof(arr) / sizeof(arr[0]);
    int i, j, temp;
```

```
printf("Original Array: ");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");

    // Bubble Sort
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {  // Compare adjacent elements
                // Swap if they are in wrong order
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
    printf("Sorted Array: ");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");

    return 0;
}
```

Output

```
Original Array: 5 2 9 1 5
Sorted Array: 1 2 5 5 9
```

Lab Tasks

Task 1: A given array is [1, 2, 3, 4, 5]. Write a C program to perform following modifications;

1. Delete the element 3 from the array.
2. Insert the element 6 at the position where 3 was removed.
3. Append the elements [7, 8, 9, 10] to the end of the array.
4. Display the array after each operation.

Code

Open task1.c ~/Desktop/PF_Labs/Lab07_2025_EE_016

```c
#include <stdio.h>

void printArray(int arr[], int n, char* message) {
    printf("%s: ", message);
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {

    int arr[10] = {1, 2, 3, 4, 5};
    int n = 5; // number of active elements

    printArray(arr, n, "Initial array");

    int del_index = 2;
    for (int i = del_index; i < n - 1; i++) {
        arr[i] = arr[i + 1]; // shift everything left
    }
    n--; // reduce active elements
    printArray(arr, n, "After deleting 3");


    int insert_index = 2;
    for (int i = n; i > insert_index; i--) {
        arr[i] = arr[i - 1];
    }
    arr[insert_index] = 6;
    n++;
    printArray(arr, n, "After inserting 6");
    int append_arr[4] = {7, 8, 9, 10};
    for (int i = 0; i < 4; i++) {
        arr[n] = append_arr[i];
        n++;
    }
    printArray(arr, n, "After appending [7, 8, 9, 10]");

    return 0;
}
```

Output

```
muhammad-umar@umarrsthinkpad:~/Desktop/PF_Labs/Lab07_2025_EE_016$ ./task1
Initial array: 1 2 3 4 5
After deleting 3: 1 2 4 5
After inserting 6: 1 2 6 4 5
After appending [7, 8, 9, 10]: 1 2 6 4 5 7 8 9 10
muhammad-umar@umarrsthinkpad:~/Desktop/PF_Labs/Lab07_2025_EE_016$
```

Task 2: Write a C program to check if the string is a **palindrome** (reads the same forwards and backwards). For example: “No lemon, no melon”.

Code

```
Open  task2.c
~/Desktop/PF_Labs/Lab07_2025_EE_016
#include <stdio.h>
#include <string.h>
#include <ctype.h> // For isalnum() and tolower()

int main() {
    char str[100];
    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);
    str[strcspn(str, "\n")] = '\0'; // Remove newline
    int left = 0;
    int right = strlen(str) - 1;
    int is_palindrome = 1;
  while (left < right) {
        while (left < right && !isalnum(str[left])) {
            left++;
        }
        while (left < right && !isalnum(str[right])) {
            right--;
        }
        if (tolower(str[left]) != tolower(str[right])) {
            is_palindrome = 0;
            break;
        }
        left++;
        right--;
    }

    if (is_palindrome) {
        printf("\"%s\" is a palindrome!\n", str);
    } else {
        printf("\"%s\" is not a palindrome.\n", str);
    }

    return 0;
}
```

Output

```
muhammad-umar@umarrsthinkpad: ~/Desktop/PF_Labs/Lab07_2025_EE_016
muhammad-umar@umarrsthinkpad:~/Desktop/PF_Labs/Lab07_2025_EE_016$ ./task2
Enter a string: No lemon No melon
"No lemon No melon" is a palindrome!
muhammad-umar@umarrsthinkpad:~/Desktop/PF_Labs/Lab07_2025_EE_016$
```

Task 3: Write a C program that stores the marks of students in an integer array from the user and sort them in **descending order** by using bubble sort algorithm.

Code

```c
#include <stdio.h>
int main() {
    int n, i, j, temp;
    printf("Enter the number of students: ");
    scanf("%d", &n);
    int marks[n];
    printf("Enter the marks of %d students:\n", n);
    for (i = 0; i < n; i++) {
        printf("Student %d: ", i + 1);
        scanf("%d", &marks[i]);
    }
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - 1 - i; j++) {
            if (marks[j] < marks[j + 1]) {
                temp = marks[j];
                marks[j] = marks[j + 1];
                marks[j + 1] = temp;
            }
        }
    }
    printf("\nMarks in descending order:\n");
    for (i = 0; i < n; i++) {
        printf("%d ", marks[i]);
    }
    printf("\n");
    return 0;
}
```

Output

```
muhammad-umar@umarrsthinkpad:~/Desktop/PF_Labs/Lab07_2025_EE_016$ ./task3
Enter the number of students: 5
Enter the marks of 5 students:
Student 1: 78
Student 2: 24
Student 3: 99
Student 4: 68
Student 5: 100

Marks in descending order:
100 99 78 68 24
muhammad-umar@umarrsthinkpad:~/Desktop/PF_Labs/Lab07_2025_EE_016$
```

Task 4: Write a C program that passes an array to a function to print only prime numbers in the given array.

Code

```
Open  task4.c  ~/Desktop/PF_L... 07_2025_EE_016

#include <stdio.h>

int isPrime(int num) {
    if (num <= 1) return 0;
    for (int i = 2; i * i <= num; i++) {
        if (num % i == 0) return 0;
    }
    return 1;
}
void printPrimes(int arr[], int size) {
    printf("Prime numbers in the array: ");
    for (int i = 0; i < size; i++) {
        if (isPrime(arr[i])) {
            printf("%d ", arr[i]);
        }
    }
    printf("\n");
}
int main() {
    int n;
    printf("Enter the number of elements: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter %d elements:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    printPrimes(arr, n);
    return 0;
}
```

Output

```
muhammad-umar@umarrsthinkpad:~/Desktop/PF_Labs/Lab07_2025_EE_016$ ./task4
Enter the number of elements: 5
Enter 5 elements:
2
3
7
10
9
Prime numbers in the array: 2 3 7
muhammad-umar@umarrsthinkpad:~/Desktop/PF_Labs/Lab07_2025_EE_016$
```

Task 5: Debug the given C code and write the corrected code with the expected output.

Bugged Code

```c
#include <stdio.h>
int main() {
    static int arr = {5, 2, 9, 'A', 1};
    int n = sizeof(arr);
    int i, j, temp;
    for(i = 0; i <= n; i++) {
        for(j = 0; j <= n; j++) {
            if(arr[j] > arr[j+1]) {
                temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
    printf("Sorted array: ");
    for(i = 0; i <= n; i++)
        printf("%d ", arr[i]);
    return 0;
}
```

Corrected Code

```c
#include <stdio.h>

int main() {
    static int arr[] = {5, 2, 9, 4, 1};
    int n = sizeof(arr) / sizeof(arr[0]);
    int i, j, temp;

    for(i = 0; i < n - 1; i++) {
        for(j = 0; j < n - 1 - i; j++) {
            if(arr[j] > arr[j+1]) {
                temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }

    printf("Sorted array: ");
    for(i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    return 0;
}
```

Output

```
muhammad-umar@umarrsthinkpad:~/Desktop/PF_Labs/Lab07_2025_EE_016$ ./task5
Sorted array: 1 2 4 5 9
muhammad-umar@umarrsthinkpad:~/Desktop/PF_Labs/Lab07_2025_EE_016$
```